# Lab 5 — Hash Maps

## Course Context

This lab is part of the Data Structures and Algorithms sequence and focuses on the design and application of hash-based data structures for efficient computation.

---

## 1. Learning Objectives

Upon successful completion of this lab, students will be able to:

- Explain the theoretical foundations of hashing and hash functions
- Analyze average-case vs worst-case time complexity of hash-based operations
- Apply hash maps to solve lookup, counting, and complement-search problems
- Design and implement linear-time algorithms using hash maps
- Solve canonical problems involving hash-based optimization strategies

---

## 2. Theoretical Foundations

### 2.1 Hashing Overview

Hashing is a technique used to map data of arbitrary size to fixed-size values. A **hash function** transforms a key into an index within an array structure.

Formally:

h(k) → [0, m-1]

Where: - k is the key - m is the size of the hash table

The resulting structure is called a **hash table**, which stores key–value pairs.

---

### 2.2 Hash Map Abstract Data Type (ADT)

A hash map supports the following operations:

- Insert(key, value)
- Delete(key)
- Search(key)

Average time complexity: - Insert: O(1) - Search: O(1) - Delete: O(1)

Worst-case complexity (due to collisions): - O(n)

---

### 2.3 Collision Handling

A **collision** occurs when two distinct keys map to the same index.

Common collision resolution strategies include:

1. **Chaining** (linked lists or dynamic arrays)
2. **Open Addressing**
    - Linear probing
    - Quadratic probing
    - Double hashing

---

### 2.4 Load Factor

The **load factor (α)** is defined as:

α = n / m

Where: - n = number of stored elements - m = table size

A high load factor increases collision probability and degrades performance.

---

### 2.5 Practical Implementations

Modern programming languages provide optimized hash map implementations:

- Java: HashMap
- Python: dict
- C++: unordered_map

These implementations internally manage resizing, hashing, and collision resolution.

---

## 3. Part I — Fundamental Operations

### 3.1 Objective

Develop familiarity with hash map operations and behavior.

### 3.2 Task

Create a mapping between student identifiers and names.

*Requirements*

- Insert at least three key–value pairs
- Retrieve a value using a key
- Update an existing value
- Remove a key from the map

*Example (Java)*

```java
import java.util.HashMap;

public class BasicHashMapExample {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();

        // Insert
        students.put(101, "Alice");
        students.put(102, "Bob");
        students.put(103, "Charlie");

        // Access
        System.out.println(students.get(101));

        // Update
        students.put(102, "Robert");

        // Delete
        students.remove(103);
    }
}
```

---

## 4. Part II — Frequency Counting Pattern

### 4.1 Objective

Apply hash maps for counting occurrences in linear time.

### 4.2 Problem Statement

Given an array of integers, compute the frequency of each element.

*Input*

[1, 2, 2, 3, 1, 1, 4]

*Output*

{1: 3, 2: 2, 3: 1, 4: 1}

---

### 4.3 Implementation Template (Java)

```java
import java.util.HashMap;

public class FrequencyCount {
    public static HashMap<Integer, Integer> frequencyCount(int[] nums) {
        HashMap<Integer, Integer> freq = new HashMap<>();

        for (int num : nums) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        return freq;
    }
}
```

---

## 5. Part III — Duplicate Detection

### 5.1 Objective

Utilize hash-based membership testing to detect duplicates efficiently.

### 5.2 Problem Statement

Given a list of integers, return the first duplicate element encountered.

*Example*

Input: [3, 1, 4, 2, 5, 3] Output: 3

---

### 5.3 Implementation Template (Java)

```java
import java.util.HashSet;

public class FirstDuplicate {
    public static Integer firstDuplicate(int[] nums) {
        HashSet<Integer> seen = new HashSet<>();
```

```
        for (int num : nums) {
            if (seen.contains(num)) {
                return num;
            }
            seen.add(num);
        }

        return null;
    }
}
```

---

## 6. Part IV — Two Sum Problem

### 6.1 Objective

Design a linear-time algorithm using hash maps to solve a complement-search problem.

### 6.2 Problem Statement

Given an array of integers and a target value, return indices of the two numbers such that they add up to the target.

---

### 6.3 Algorithmic Strategy

Maintain a hash map that stores:

value → index

At each iteration:

- Compute complement = target − current value
- Check if complement exists in the map
- If yes, return indices
- Otherwise, insert current value

---

### 6.4 Implementation (Java)

```
import java.util.HashMap;

public class TwoSum {
    public static int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();
```

```
        for (int i = 0; i < nums.length; i++) {

            // add missing codes here

            map.put(nums[i], i);
        }

        return new int[]{};
    }
}
```

---

## 7. Part V — Roman to Integer

### 7.1 Objective

Apply hash maps for symbolic mapping and conditional accumulation.

### 7.2 Problem Statement

Convert a Roman numeral string into its integer representation.

---

### 7.3 Symbol Mapping

I → 1 V → 5 X → 10 L → 50 C → 100 D → 500 M → 1000

---

### 7.4 Algorithmic Insight

Traverse the string from left to right:

- If current value < next value → subtract
- Otherwise → add

---

### 7.5 Implementation (Java)

```
import java.util.HashMap;

public class RomanToInteger {
    public static int romanToInt(String s) {
        HashMap<Character, Integer> values = new HashMap<>();
        values.put('I', 1);
        values.put('V', 5);
        values.put('X', 10);
```

```
        values.put('L', 50);
        values.put('C', 100);
        values.put('D', 500);
        values.put('M', 1000);

        int total = 0;

        for (int i = 0; i < s.length(); i++) {
            // add missing codes here
        }

        return total;
    }
}
```

---

## 8. Part VI — Complexity Analysis

| Operation | Time Complexity |
|---|---|
| Hash Map Insert | O(1) average |
| Hash Map Lookup | O(1) average |
| Frequency Count | O(n) |
| Two Sum | O(n) |
| Roman to Integer | O(n) |

---

## 9. Submission Requirements

Students must submit:

1. **Source code for all tasks**
2. **Output screenshots or logs**
3. **Brief complexity analysis for Two Sum and Roman to Integer**

---

## 10. Evaluation Criteria

- Correctness of implementation
- Adherence to required time complexity
- Code clarity and structure
- Proper use of hash maps

---

## 11. Optional Extensions

- Implement a basic hash table from scratch
- Solve additional problems:
    - Longest Substring Without Repeating Characters
    - Group Anagrams

---

## 12. Summary

This lab introduces hash maps as a fundamental tool for designing efficient algorithms. Through progressive tasks, students develop the ability to reduce computational complexity from quadratic to linear time using appropriate data structures.